AI AGENTS · HOME CLUSTER · K3S

Hermes, in the cluster
an autonomous agent with its own node, its own sandbox, its own traces

The Nous Research **Hermes Agent** now runs as a first-class cluster service on the brand-new CPU-only node **x1** — one pod that serves both an OpenAI-compatible API and a web dashboard, does all its thinking through the **LiteLLM gateway**, and streams every agent step into **Arize Phoenix**. The headline: the official image already ships Node 22, ffmpeg 7.1 and Playwright Chromium, so the **HyperFrames video-render stack works inside the agent's sandbox with zero extra image work** — verified today with a 300-frame render in ~43 s. Closes Forgejo issue **brian/home-lab#10**.

Image nousresearch/hermes-agent:v2026.6.19 | **Namespace** hermes (pinned to x1) | **Dashboard** https://hermes.lan/

API https://hermes.lan/v1 (bearer) | **Manifests** clusters/home/hermes/

x1
NEW NODE
CPU-only · 8c / 7.6 Gi / ~250 G · 192.168.5.174

43s
300 FRAMES
HyperFrames smoke.mp4 rendered in-pod on x1's CPU

$25
30-DAY BUDGET
Dedicated LiteLLM virtual key, alias hermes-agent

30Gi
STATE PVC
local-path at /opt/data — config, skills, memory, sessions

The core mental model: two sandboxes, not one

Hermes executes real shell commands as a tool. Where those commands land is the whole security story:

- **Outer sandbox = the pod.** Kubernetes-native isolation: its own filesystem (image + PVC), its own cgroup limits (5 Gi memory cap), its own network identity. If the agent trashes anything, it trashes the pod and the PVC — `kubectl delete pod` is the reset button.
- **Inner sandbox = `terminal.backend`.** Set to `local`, so tool commands run *inside the container*. The alternative backends: `docker` would mean Docker-in-Docker inside k8s (avoided, deliberately), and `ssh` is the future path — point the agent at a chosen "body" host when you actually want it to control a real machine.

Unlearn: "an agent container needs Docker for sandboxing" (and "AI needs a GPU")

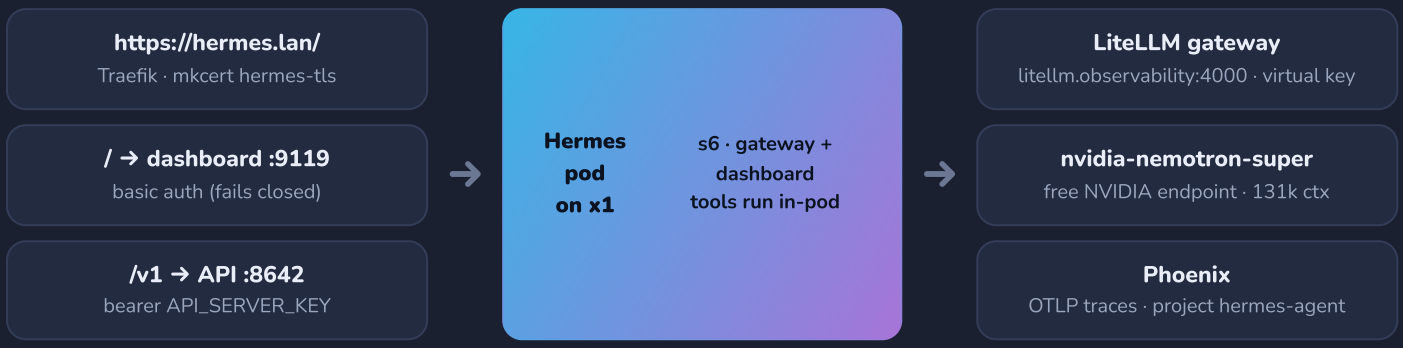
On bare metal, Hermes' Docker backend is the sandbox. In Kubernetes the **pod already is one** — adding Docker inside it buys you DinD complexity, privileged containers, and a second image cache for nothing. And Hermes itself is a Python brain that calls remote LLMs: it never touches a GPU, which is exactly why it lives on the cluster's first **CPU-only** node instead of taking up residence next to a 4090.

The a3 bare-metal Hermes was left alone

A pre-existing Hermes install on a3 (`~/.hermes`, gateway running) is untouched — the k8s deployment is a parallel, separately-stated instance. Retire-or-keep is a listed follow-up, decided after the cluster instance proves itself.

Anatomy: one pod, one container, two services

The official image uses **s6-overlay as PID 1**: one container supervises both the gateway (OpenAI-compatible API on **:8642**) and the dashboard (**:9119**, switched on with `HERMES_DASHBOARD=1`). The container **starts as root on purpose** — the s6 stage2 hook chowns `/opt/data` and seeds config, then drops every service to the internal `hermes` user (uid 10000). Don't fight it with `runAsUser`.



PIECE	WHAT / WHERE	WHY IT'S LIKE THAT
Deployment	1 replica, `strategy: Recreate`	Single writer over the sqlite-ish session/memory stores on the PVC — never two pods on one `/opt/data`
PVC	30 Gi `local-path` → `/opt/data`	All agent state (config, skills, memory, sessions); the image stays stateless. Longhorn later for mobility
Seed ConfigMap	`hermes-seed` → initContainer copy	Copied to `/opt/data/config.yaml` **only if absent** — after first boot the PVC copy is live (Hermes writes to it)
Resources	req 500m/1 Gi, limit 5 Gi mem	Headroom for in-pod tool runs (Chromium + ffmpeg renders); x1 has 7.6 Gi total
Probes	`/health` on :8642	Startup probe tolerates 5 min of first-boot seeding before liveness takes over
Homepage / Gatus	AI group tile · 2 health checks	Auto-discovered via ingress annotations; Gatus probes both ports (page 4)

Auth: both doors are locked

Dashboard — fails closed by design

Since the June 2026 hardening, the dashboard **refuses a non-loopback bind without an auth provider** (`HERMES_DASHBOARD_INSECURE` is a deprecated no-op). The bundled `basic` provider activates via env: username `brian`, password + HMAC session secret from the `hermes-secrets` k8s secret.

API — treat it like root

`/v1` requires the bearer `API_SERVER_KEY` (verified: unauthenticated request → **401**). Remember what this API *is*: a chat endpoint that can **execute tools in the pod**. It stays LAN-only — no Tailscale ingress for this one.

All four secret values (`hermes-api-server-key`, `litellm-key-hermes-agent`, `hermes-dashboard`, `hermes-dashboard-session-secret`) live in the Vaultwarden **Automation** collection and land in-cluster as `hermes-secrets` via `scripts/vault-secret.sh` — nothing in git (public repo). The exact `kubectl create secret` incantation is in the Deployment's header comment.

Inference: everything through LiteLLM

Hermes never talks to a model provider directly. A `custom_providers` block points it at the in-cluster LiteLLM gateway with a **dedicated virtual key** (alias `hermes-agent`, **$25 / 30-day budget**) — so the agent gets one bill line, one kill switch, and its usage shows up in LiteLLM spend like any other client. Default model is **nvidia-nemotron-super** (free NVIDIA endpoint, 131,072-token context); switch any time with `/model custom:litellm:<name>`.

```
# seed config.yaml (clusters/home/hermes/configmap.yaml) — the interesting bits
custom_providers:
  - name: litellm
    base_url: http://litellm.observability.svc.cluster.local:4000/v1
    key_env: LITELLM_API_KEY          # from hermes-secrets, never in the file
    models: { nvidia-nemotron-super: {context_length: 131072}, groq-llama-3.3-70b: {…} }
model:    { default: nvidia-nemotron-super, provider: custom:litellm }
terminal: { backend: local }              # the pod IS the sandbox
plugins:  { enabled: [hermes_otel] }
```

Observability: hermes-otel → Phoenix

Traces come from Brian's own plugin, **hermes-otel** (`github.com/briancaffey/hermes-otel`), installed with `hermes plugins install briancaffey/hermes-otel --enable` into `/opt/data/plugins/hermes_otel`. It exports OTLP/HTTP to Arize Phoenix (`phoenix-svc.observability:6006/v1/traces`) under a dedicated Phoenix project — set by `OTEL_PHOENIX_ENDPOINT` + `OTEL_PROJECT_NAME`.

Project `hermes-agent`

The agent's own view: verified span tree `agent → llm.nvidia-nemotron-super → api.nvidia-nemotron-super`. This is where a future "make me a video" run will show skill → terminal → render steps.

Project `default`

LiteLLM's gateway-side traces, untouched. Same request shows up in both projects at matching timestamps — agent view and gateway view of one call, cleanly separated.

The wrinkle: the image's venv doesn't bundle OpenTelemetry

`/opt/hermes/.venv` is **root-owned and read-only at runtime** (uv-managed, no pip, lazy installs disabled) — without the OTel SDK the plugin silently degrades to NoopSpan. Two-stage fix:

- **Interim (running now):** an initContainer does `uv pip install --target /opt/data/otel-libs` for the three OTel packages, and the main container sets `PYTHONPATH=/opt/data/otel-libs`. Ugly, works, self-heals on PVC.
- **Final (built & pushed):** a derived image — `clusters/home/hermes/image/Dockerfile`, already at `harbor.lan/library/hermes-agent:v2026.6.19-otel` — bakes the libs into the venv with `uv pip install --python /opt/hermes/.venv/bin/python`. Blocked only on x1 trusting the Harbor CA: one-time `sudo scripts/trust-harbor-ca.sh` on x1, then swap the image and delete the initContainer.

The headline: HyperFrames already works in the sandbox

> **The end goal is agent-driven video generation** — prompt Hermes, get an mp4. The expected hard part was the render toolchain… and it turned out to be **zero extra image work**: the official Hermes image already ships **Node 22**, **ffmpeg 7.1**, and **Playwright headless Chromium**, which is exactly the HyperFrames render stack. The agent's inner sandbox is a fully-equipped render box out of the box.

Verified today, in-pod on x1

```
# 1. install the skill through Hermes' own skill hub (lands on the PVC, survives restarts)
hermes skills install skills-sh/hyperframes     # → ~/.hermes/skills/creative/hyperframes

# 2. smoke-test the raw toolchain inside the container
npx hyperframes init smoke
npx hyperframes render                              # → smoke.mp4
#    300 frames rendered in ~43 s on x1's CPU — Chromium capture + ffmpeg encode
```

Next experiment: prompt Hermes end-to-end ("make me a video about…") and watch the skill → terminal → render pipeline appear as a span tree in Phoenix. The 5 Gi memory limit exists precisely for these Chromium+ffmpeg tool runs.

Test results — verified end-to-end

CHECK	WHAT WAS DONE	RESULT
Full inference path	`POST https://hermes.lan/v1/chat/completions` → Hermes → LiteLLM → nvidia-nemotron-super	PASS correct reply ("HERMES-ON-K3S-OK")
API auth	Same endpoint, no bearer token	PASS 401 rejected
Tracing, both sides	Phoenix project list + spans	PASS `hermes-agent` + `default`, spans at matching timestamps
Render stack	hyperframes init + render in-pod	PASS smoke.mp4, 300 frames / ~43 s
Monitoring	Gatus: `http://hermes.hermes:8642/health` + `:9119/api/status`	PASS both UP, email alerts wired
Dashboard tile	Homepage auto-discovery via ingress annotations	PASS tile in the AI group

A fun datapoint: what an agent costs per "hello"

The verification chat consumed **~27,900 prompt tokens** — that's the full Hermes agent system prompt (tool specs, skills, memory scaffolding) riding along on every turn. On the free NVIDIA endpoint that's fine; it's also exactly why the agent got a **budgeted** LiteLLM key before it got access to anything that bills per token.

Gotchas — future-you will hit these

1. **A Service named `hermes` poisons the env** `breaks boot`

   Kubernetes service links inject `HERMES_PORT=tcp://…` and friends into the pod — which collide with the image's own `HERMES_*` config parsing. Fix: `enableServiceLinks: false` on the pod spec. Applies to any app whose env-var prefix matches its Service name.

2. **Traefik v3 502s multi-GB pushes to harbor.lan** `registry`

   Default `readTimeout=60s` kills any large layer upload through the ingress. Workaround used today: `kubectl port-forward` to harbor-core and `crane push` to localhost. Long-term: raise the entrypoint timeout for Harbor's route.

3. **vault-secret.sh substring matching vs prefix-named items** `secrets`

   `bw` matches item names by substring, so "hermes-dashboard" also matches "hermes-dashboard-session-secret" → "More than one result" → an **empty string** inside `$(…)`, and you create a secret with a blank value. Fetch by item ID, or never name one item as a prefix of another.

4. **The hermes venv has no pip** `images`

   It's uv-managed. Derived images must install with `uv pip install --python /opt/hermes/.venv/bin/python …` — a plain `pip install` layer fails.

5. **Skill installs go through Hermes' hub, not npm** `skills`

   It's `hermes skills install <source>` (e.g. `skills-sh/hyperframes`), not `npx skills add`. Installed skills land under `~/.hermes/skills/` on the PVC.

Operating it

```
# deploy / update (kustomize service — the usual way)
kubectl apply -k clusters/home/hermes

# runtime config lives on the PVC, not in git — edit via dashboard or:
kubectl -n hermes exec deploy/hermes -- cat /opt/data/config.yaml

# logs (s6 multiplexes gateway + dashboard into the one container)
kubectl -n hermes logs deploy/hermes -f

# nuke-and-repave the agent's world (config re-seeds from the ConfigMap)
kubectl -n hermes delete pvc hermes-data   # + delete pod
```

Remember the seeding rule: `configmap.yaml` only matters on **first boot** (or after deleting the PVC copy). Day-2 model switches, plugin toggles and dashboard edits all mutate the live `/opt/data/config.yaml` — the repo file is the birth certificate, not the passport.

Where this goes next

1. **Cluster control — a scoped ServiceAccount + RBAC**

   Let Hermes operate the cluster the way Claude does, starting **read-only** (get/list/watch) and widening verb-by-verb as trust builds. The outer sandbox already contains the blast radius; RBAC defines the reach.

2. **Swap to the Harbor `-otel` image**

   One `sudo scripts/trust-harbor-ca.sh` on x1, then drop the initContainer + PYTHONPATH hack for the already-pushed `harbor.lan/library/hermes-agent:v2026.6.19-otel`.

3. **Harden the cage**

   NetworkPolicy (egress allow-list: LiteLLM, Phoenix, DNS), `skills.write_approval` if the agent starts authoring its own skills, and PVC backup once Longhorn lands (#13) — the 30 Gi of memory/sessions is the one thing that can't be re-seeded.

4. **More surfaces, fewer duplicates**

   Telegram gateway for chat-from-anywhere; a deliberate `ssh` terminal backend when the agent should drive a real "body" host; and a retire-or-keep decision on the a3 bare-metal Hermes once the cluster one has a few weeks of uptime.

Bottom line

Hermes went from a bare-metal experiment on a3 to a **properly caged, budgeted, observable cluster citizen**: pod-as-sandbox on a CPU-only node, all inference metered through a $25/30d LiteLLM key, every agent step traced into its own Phoenix project, both entry points authenticated, health-checked by Gatus, state on a 30 Gi PVC that survives any restart. And the thing that was supposed to be the hard part — a video-render toolchain for HyperFrames — **shipped in the base image**. The next milestone isn't infrastructure; it's a prompt: "make me a video about…", watched live in Phoenix.

Quick reference

THING	VALUE
Dashboard / API	`https://hermes.lan/` (basic auth) · `https://hermes.lan/v1` (bearer, LAN-only)
Manifests	`clusters/home/hermes/` — `kubectl apply -k`
Node	**x1** · 192.168.5.174 · 8c / 7.6 Gi · `inference-club.com/box=x1`
Model default	`nvidia-nemotron-super` via LiteLLM · switch: `/model custom:litellm:<name>`
Secrets	Vaultwarden Automation → k8s secret `hermes-secrets` (4 keys)
Traces	Phoenix project `hermes-agent` (`phoenix-svc.observability:6006`)
State	PVC `hermes-data` 30 Gi local-path → `/opt/data`
Health	Gatus: `hermes.hermes:8642/health` + `:9119/api/status`
Derived image	`harbor.lan/library/hermes-agent:v2026.6.19-otel` (pending x1 CA trust)